Язык программирования Python

Списки, кортежи

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

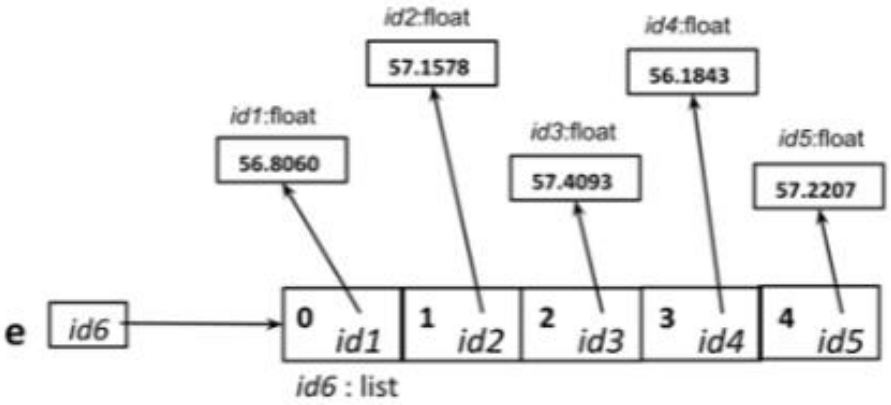
Списки и кортежи

Для хранения наборов данных в Python часто используют **кортежи (tuple)** или **списки (list)**, которые позволяют хранить произвольное количество объектов любого типа в виде упорядоченной последовательности.

Разница между ним в том, что **кортеж – неизменяемый**, а **список – изменяемый**.

```
>>> e = [56.8060, 57.1578, 57.4093, 56.1846, 57.2207]
>>> e
[56.8060, 57.1578, 57.4093, 56.1846, 57.2207]
```

Фактически, список и кортеж хранят ссылки на объекты других типов.



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Создание кортежа

Кортеж создаётся с помощью круглых скобок ( ) и запятых, отделяющих элементы кортежа друг от друга, или функцией tuple().

Например, так выглядит создание кортежа с присваиванием ему конкретных значений:

```
t1 = (42,) # Кортеж из одного элемента.Запятая обязат.
t2 = (1, 34, 56) # Кортеж целых чисел
t3 = (1.12, 3.141, 65.2) # Кортеж вещественных чисел
t4 = ('Дарья', 'Михаил', 'Владимир') # Кортеж строк
t5 = ('Москва', 1603) # Кортеж из смешанных типов
t6 = 1, 2, 3 # Скобки можно опускать
```

Также элементами кортежа могут являться другие коллекции, в том числе и кортежи:

```
# Кортеж, состоящий из двух кортежей
t6 = (('Иван', 1), ('Фёдор', 2))
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Создание списка

Список создаётся с помощью квадратных скобок [ ] и запятых, отделяющих элементы списка друг от друга, или функцией list().

Например, так выглядит создание пустого списка:

```
s1 = list() # Пустой список
s2 = [] # Пустой список
```

Списку сразу можно присвоить конкретные значения:

```
s3 = [1, 34, 56] # Список целых чисел
s4 = [1.12, 3.141, 65.2] # Список вещественных чисел
s5 = ['Дарья', 'Михаил', 'Владимир'] # Список строк
s6 = ['Москва', 1603] # Список из смешанных типов
```

Элементами списка могут являться другие коллекции:

```
# Список, состоящий из кортежа и списка
s7 = [('Иван', 1), ['Фёдор', 2]]
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обращение к элементам. Индекс

Каждый элемент списка или кортежа имеет собственный порядковый номер (**индекс**). Нумерация начинается с нуля. Обратиться к отдельному элементу можно записав имя списка и в квадратных скобках индекс этого элемента:

```
>>> s = [1, 'Hi', 42, 'O', 7, 'p', 3]
>>> s[0]
1
>>> s[1]
'Hi'
```

1	Hi	42	O	7	p	3
0	1	2	3	4	5	6

Для обращения к элементам также могут применяться отрицательные индексы. В этом случае отсчёт ведётся с конца списка:

```
>>> s = [1, 'Hi', 42, 'O', 7, 'p', 3]
>>> s[-1]
3
>>> s[-2]
'p'
```

1	Hi	42	O	7	p	3
-7	-6	-5	-4	-3	-2	-1

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обращение к элементам. Размер

Для вычисления индекса используется формула: `размер списка + отрицательный индекс`. Размер (количество элементов) можно найти с помощью функции **len()**, которой передаётся список в качестве аргумента:

```
>>> s = [1, 'Hi', 42, 'O', 7, 'p', 3]
>>> len(s)
7
>>> s[len(s)-1]
'!'
```

1	Hi	42	O	7	p	3
0	1	2	3	4	5	6

Обращение по несуществующему индексу приведёт к ошибке:

```
>>> s[100]
Traceback (most recent call last):
  File "<pyshell#145>", line 1, in <module>
    s[100]
IndexError: list index out of range
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обращение к элементам. Срез (1)

Если вам необходимо выбрать только часть списка, то для этого можно применять **срезы**:

```
>>> s = [1, 'Hi', 42, 'O', 7, 'p', 3]
>>> s
[1, 'Hi', 42, 'O', 7, 'p', 3]
>>> s[1:4:1]
['Hi', 42, 'O']
```

1	Hi	42	O	7	p	3
0	1	2	3	4	5	6

Срез – это специальная операция, которая создаёт новый список из части исходного списка. При этом внутри квадратных скобок через двоеточие нужно указать три значения:

1. индекс начала (по умолчанию 0);
2. индекс конца (не включительно!) (по умолчанию последний);
3. шаг выборки (по умолчанию 1).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обращение к элементам. Срез (2)

При использовании среза не обязательно указывать все три параметра. Допустимо некоторые из них опускать.

Например:

1	Hi	42	O	7	p	3
0	1	2	3	4	5	6

```
>>> s = [1, 'Hi', 42, 'O', 7, 'p', 3]
>>> s[2:5] # со 2-го индекса по 5-ый с шагом 1
[42, 'O', 7]
>>> s[:3] # с 0-го индекса по 3-ий с шагом 1
[1, 'Hi', 42]
>>> s[:] # весь список (создаётся копия)
[1, 'Hi', 42, 'O', 7, 'p', 3]
>>> s[::2] # с 0-го индекса до конца с шагом 2
[1, 42, 7, 3]
>>> s[::-1] # с конца до 0-го индекса с шагом -1 (переворот)
[3, 'p', 7, 'O', 42, 'Hi', 1]
>>> s = ['T'] + s[2:] # со 2-го индекса до конца с шагом 1
>>> s
['T', 42, 'O', 7, 'p', 3]
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Присваивание новых значений

Отдельным элементам списка, в отличие от кортежа или строки, можно присваивать новые значения:

```
>>> s = [1, 'Hi', 42, 'O', 7, 'p', 3]
>>> s[0] = 'New'
>>> s[-1] = 7
>>> s
['New', 'Hi', 42, 'O', 7, 'p', 7]
```

1	Hi	42	O	7	p	3
0	1	2	3	4	5	6

Аналогичным образом можно работать и со срезами. Главное помнить, что срез – это тоже список:

```
>>> s[1:4] = ['Меняем', 'три', 'элемента']
>>> s
['New', 'Меняем', 'три', 'элемента', 7, 'p', 7]
>>> s[0:0] = [1, 2]
>>> s
[1, 2, 'New', 'Меняем', 'три', 'элемента', 7, 'p', 7]
```

Выбираем пустой срез перед первым элементом

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Удаление элементов

Для удаления отдельных элементов списка (кортежи неизменяемы) используется оператор **del**. При этом меняется количество элементов и, как следствие, их индексы:

1	Hi	42	O	7	p	3
0	1	2	3	4	5	6

```
>>> s = [1, 'Hi', 42, 'O', 7, 'p', 3]
>>> del s[0]
>>> s
['Hi', 42, 'O', 7, 'p', 3]
>>> del s[0]
>>> s
[42, 'O', 7, 'p', 3]
```

Можно удалять несколько элементов с помощью срезов:

```
>>> del s[1:3]
>>> s
[42, 'p', 3]
```

42	O	7	p	3
0	1	2	3	4

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перебор элементов (1)

Для того, чтобы выполнить какие-либо операции над элементами списка, их можно перебрать с помощью цикла. Сделать это можно тремя способами.

1. Если нужно только считать значения элементов и нас не волнует их индекс, то выполняется это так:

```
>>> s = [1, 'Hi', 42]
>>> for elem in s:
        print(elem)
```

```
Ответ:
1
'Hi'
42
```

В переменную elem последовательно записываются ссылки на элементы списка.

2. Если нужно только считать значения элементов и при этом нам нужны их индексы, то выполняется это с помощью функции **enumerate()** (возвращает пару (*индекс*, *элемент*)):

```
>>> s = [1, 'Hi', 42]
>>> for i, elem in enumerate(s):
        print(i, '-', elem)
```

```
Ответ:
0 - 1
1 - 'Hi'
2 - 42
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перебор элементов (2)

3. Если нужно не только считать значения элементов, но и записать новые, то нам понадобится перебор элементов по индексу с помощью функции **range()**:

```
>>> s = [1, 5, 8]
>>> for i in range(len(s)):
        s[i] *= 2

>>> s
[2, 10, 16]
```

len() задаёт ограничение для индексов. range() генерирует последовательность индексов.

Выбирать тот или иной способ перебора элементов необходимо исходя из поставленной задачи.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Проверка наличия значения

Полезным является оператор **in**, позволяющий проверить вхождение в список какого-либо значения:

```
>>> ls = [1, 'Hi', 42, 'O', 7, 'p', 3]
>>> 1 in ls
True
>>> 2 in ls  # проверяем на вхождения
False
>>> 2 not in ls  # проверяем на отсутствие
True
```

Как правило он используется совместно с оператором if . Также может применяться с оператором while.

```
>>> if 'Hi' in ls:
        print('Значение есть в списке')
'Значение есть в списке'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Операции над списками

В Python встроен набор функций и операций. Некоторые из них часто используют совместно со списками и кортежами.

С помощью операции **сложения** в Python можно объединить несколько списков в один новый. В коде для этого используется символ «+», а также «+=» если нужно добавить к существующему:

```
>>> s = [1, 2] + [3, 4]
>>> s
[1, 2, 3, 4]
>>> s += [42]
>>> s
[1, 2, 3, 4, 42]
```

Складывать можно только списки! Список и, например, число сложить не получится.

Также можно создать список из заданного количества **повторов** исходного списка. Для этого используется операция «*»:

```
>>> s * 2
[1, 2, 3, 4, 42, 1, 2, 3, 4, 42]
```

На первом месте может быть и список, и число.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Функции для работы со списками

Если в последовательности хранятся только элементы, которые можно сравнивать друг с другом (например, только числа или только строки), то возможно использование следующих функций:

```
t = (1, 9, 5, 7, 2)
print(max(t))
st = ['b', 'a', 'c']
print(min(st))
```

```
Ответ: 9
```

```
Ответ: 'a'
```

Также в этом случае возможна **сортировка** значений. При этом всегда создаётся новый список (даже если исходным был кортеж):

```
t_sort = sorted(t, reverse=False)
print(t_sort)
```

```
Ответ: [1, 2, 5, 7, 9]
```

Если все элементы последовательности можно складывать друг с другом, то возможно нахождение суммы всех элементов:

```
print(sum(t))
```

```
Ответ: 24
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Функции для работы со списками. key

У первых трёх функций (min(), max(), sorted()) есть дополнительный параметр **key**, с помощью которого можно задать что именно будет сравниваться у элементов.

К примеру, есть список стран, представленных кортежами с названиями и количеством населения. Для сортировки по населению нужно указать, что от каждого картежа списка берётся только значение с населением.

```
>>> def cmp_countries(x):
        return x[2]
```

Функция принимает 1 элемент внешнего списка. Возвращает 3-ье значение этого элемента (население).

```
>>> countries = [('РФ', 'Москва', 147),
                 ('КНР', 'Пекин', 1401),
                 ('США', 'Вашингтон', 329)]
>>> sorted(countries, key=cmp_countries)
  # sorted(countries, key=lambda x: x[2])
[('РФ', 'Москва', 147), ('США', 'Вашингтон', 329),
('КНР', 'Пекин', 1401)]
```

Можно с помощью безымянных функций

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы списков (1)

У коллекций (в том числе у списков и кортежей) есть специальные встроенные функции, называемые **методами**, которые выполняются для конкретных последовательностей.

Методы имеют две формы вызова: полную и сокращённую.

Полная форма вызова для списка:

```
list.метод(список, аргументы)
```

Аргументы могут отсутствовать.

Сокращённая форма вызова:

```
список.метод(аргументы)
```

Для кортежа вместо list используется tuple.

Например, вызовем метод **count()**, предназначенный для нахождения количества вхождений указанного значения. В качестве параметра необходимо передать искомое значение:

```
>>> ls = [7, 9, 5, 7, 2]
>>> ls.count(7)
2
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы списков (2)

Поскольку списки являются изменяемым типом данных, то для них разработано большее количество методов.

Метод	Описание
ls.append(***x***)	Добавляет элемент ***x*** в конец списка **ls**
ls.extend(***L***)	Расширяет список **ls**, добавляя в конец все элементы из списка ***L***
ls.insert(***i***, ***x***)	Вставляет перед элементом с индексом ***i*** значение ***x***
ls.remove(***x***)	Удаляет первый элемент в списке, имеющий значение ***x***
ls.pop(***i***)	Удаляет элемент с индексом ***i*** и возвращает его значение. Если индекс не указан, удаляется последний элемент
ls.count(***v***)	Возвращает количество элементов со значением ***v***.
ls.index(***v***, ***start***, ***end***)	Возвращает индекс элемента со значением ***v***. Поиск в диапазоне [***start***, ***end***) или всём списке если не задано.
ls.reverse()	Переставляет элементы в обратном порядке
ls.copy()	Возвращает копию элементов списка
ls.clear()	Очищает список

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Демонстрация методов списка (1)

```
1  >>> a = [0, 2]
2  >>> print(a) # Исходный список a
3  [0, 2]
4  >>> b = [5, 6]
5  >>> print(b) # Исходный список b
6  [5, 6]
7  >>> x = 99
8  >>> a.append(x) # Добавляем в конец списка a значение перемен. x
9  >>> print(a)    # Можно таким образом: a += [x]
10 [0, 2, 99]
11 >>> a.extend(b) # Добавляем в конец списка a элементы списка b
12 >>> print(a)    # Можно таким образом: a += b
13 [0, 2, 99, 5, 6]
14 >>> a.insert(4, x) # Вставляем перем. x перед элем. с индексом 4
15 >>> print(a)
16 [0, 2, 99, 5, 99, 6]
17 >>> a.remove(x) # Удаляем первый элемент со значением из x
18 >>> print(a)
19 [0, 2, 5, 99, 6]
20
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Демонстрация методов списка (2)

```
1  >>> y = a.pop() # Записываем в y последний элемент списка a
2  >>> print(y)    # и удаляем его из списка
3  6
4  >>> print(a)
5  [0, 2, 5, 99]
6  >>> y = a.pop(0) # Записываем в y элемент списка a с индексом 0
7  >>> print(y)     # и удаляем его из списка
8  0
9  >>> print(a)
10 [2, 5, 99]
11 >>> a.reverse() # Меняем порядок элементов на обратный
12 >>> print(a)    # Можно таким образом: a = a[::-1]
13 [99, 5, 2]
14 >>> z = a.copy() # Создаём копию списка a
15 >>> print(z)     # Можно таким образом: z = a[:]
16 [99, 5, 2]
17 >>> z.clear() # Удаляем элементы из списка
18 >>> print(z)  # Можно таким образом: del z[:]
19 []
20
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Демонстрация методов списка (3)

Полезным является метод **index(val, start, end)**, который возвращает индекс первого вхождения значения val. Поиск идёт в диапазоне индексов [start, end). По умолчанию ведётся поиск с начала строки и до конца. Если значение не найдено, то происходит ошибка ValueError.

```
>>> ls = [7, 9, 5, 7, 2]
>>> s.index(7)
0
>>> s.index(7)  # при повторе результат не меняется
0
```

Для нахождения всех вхождений значения необходим цикл:

```
>>> val = 7   # искомое значение
>>> idx = 0   # индекс начала поиска
>>> while val in ls[idx:]:   # есть ли подходящее значение?
        idx = ls.index(val, idx)   # ищем его индекс
        print(idx)   # выводим текущий результат
        idx += 1     # смещаемся для следующего поиска

0
3
```

После первого вхождения продолжаем поиск со следующего элемента.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Функции для создания списка

Небольшие списки часто задают литералами, но длинные создаются программно.

Список последовательных целых чисел можно получить с помощью функции range():

```
>>> ls = list(range(1, 10))
>>> ls
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Если нужно создать новый список путём редактирования уже существующего, то часто применяется функция map(). Она принимает два параметра: имя функции, которая будет применяться к каждому элементу исходного списка, и сам этот список.

Возведём в куб каждый элемент списка ls:

```
>>> ls = list(map(lambda x: x**3, ls))
>>> ls
[1, 8, 27, 64, 125, 216, 343, 512, 729]
```

lambda-функции чаще всего используются в таких местах.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Создание списка циклом

Для создания более сложных списков часто используется цикл for … in. Например, нам нужно получить список високосных годов в диапазоне от 1970 до 2000:

```
>>> ls = []
>>> for y in range(1970, 2001):
        if y%400 == 0 or (y%4 == 0 and y%100 != 0):
            ls.append(y)

>>> ls
[1972, 1976, 1980, 1984, 1988, 1992, 1996, 2000]
```

Каждый кратный 400, а также кратный 4, но не кратный 100.

append() добавляет элемент в конец списка.

Такой способ рабочий, но он создаёт (или переписывает) переменную **y**, которая остаётся в программе с последним сохранённым значением.

Чтобы избавиться от такого побочного эффекта можно использовать генераторы списков.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Списковое включение

Списковое включение – это выражение и цикл с дополнительным условием, заключённые в квадратные скобки. При этом цикл используется для создания элементов списка, а условие – для фильтрации нужных элементов.

Общий вид спискового включения:

```
список = [выражение for переменная in итерир_выражение
          if условие]
```

При необходимости часть (`if условие`) можно опустить. Тогда выбираются все переменные.

Что эквивалентно следующему коду:

```
список = []
for переменная in итерир_выражение:
    if условие:
        список.append(выражение)
```

Задачу с прошлого слайда можно переписать в две строки. Обратите внимание, что запись цикла for и условия if остались неизменны.

```
>>> ls = [y for y in range(1970, 2001)
          if y%400 == 0 or (y%4 == 0 and y%100 != 0)]
>>> ls
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Демонстрация спискового включения

Другие примеры:

```
>>> vec = [-4, -2, 0, 2, 4]
>>> [x*2 for x in vec]  # удваиваем значения
[-8, -4, 0, 4, 8]
>>> [x for x in vec if x >= 0]  # выбираем больше нуля
[0, 2, 4]
>>> [abs(x) for x in vec]  # применяем функцию
[4, 2, 0, 2, 4]
```

Перепишем поиск индексов вхождений определённого значения в список через списковое включение (сравните со слайдом 22):

```
>>> ls = [7, 9, 5, 7, 2]
>>> val = 7   # искомое значение
>>> idx = [i for i in range(len(ls)) if ls[i] == val]
>>> idx
[0, 3]
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Заполнение с клавиатуры

Создать и заполнить список с клавиатуры можно расписав всё через цикл:

```
>>> vec = []
>>> n = int(input())
>>> for _ in range(n):
        elem = float(input())
        vec.append(elem)

>>> vec
```

Цикл нужен только для срабатывания определённого количества раз.
Сама переменная нас не интересует, поэтому вместо неё используем _

А можно через списковое включение:

```
>>> ls = [float(input()) for i in range(int(input()))]
2
4
5
>>> ls
[4.0, 5.0]
```

По сути, всё тоже самое, но в одной строке. Для ясности можно вынести запрос на количество элементов в отдельную строку.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Многомерные списки

Часто для хранения информации необходима таблица. Программно она выглядят как список со вложенными списками. Для обращения к отдельному элементу нужно указать два индекса:

спск[*индекс вложенного списка*][*индекс элемента во вложенном списке*]

```
>>> tab = [[7, 9], [5, 7], [2, 1]]
>>> tab[0][1] = 42
>>> tab
[[7, 42], [5, 7], [2, 1]]
```

В первом вложенном списке меняем второй элемент.

Для перебора всех элементов нужно два цикла:

```
>>> for row in tab:    # выбираем вложенный список
        for elem in row:    # выбираем элемент из него
            print(elem, end=' ')    # выводим этот элемент
        print()    # переводим на новую строку

7 42
5 7
2 1
```

Для перебора можно применять те же способы, что и для одномерных (см. слайды 12-13).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования вложенных последовательностей

По введенным числу, месяцу и году определить порядковый номер дня:

```
# заполнение кортежа, хранящего кол-во дней в месяцах
tab_days = ((0,31,28,31,30,31,30,31,31,30,31,30,31),
            (0,31,29,31,30,31,30,31,31,30,31,30,31))
year = int(input('Введите год: '))
month = int(input('Введите месяц: '))
day = int(input('Введите число: '))
# определим, високосный ли год
vis = int(year%400 == 0 or (year%4 == 0 and year%100 != 0))
if vis == 1:
    print('Год високосный')
else:
    print('Год не високосный')
# цикл по номеру месяца
for m in range(1, month):
    # С помощью vis выбираем один
    # из вложенных кортежей. m – индекс элемента этого корт.
    day +=  tab_days[vis][m]
print('В ', year, ' году этот день имеет номер ', day)
```

Каждый кратный 400, а также кратный 4, но не кратный 100.

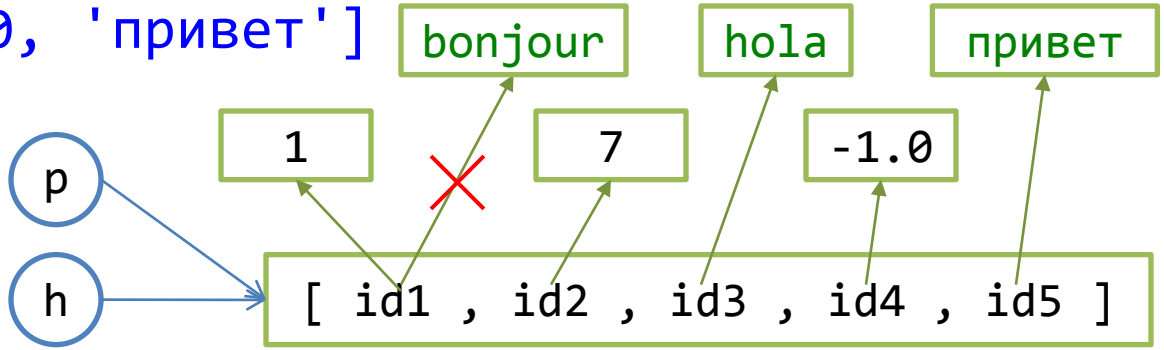
```
Введите год: 2020
Введите месяц: 3
Введите число: 10
Год високосный
В  2020  году этот день
имеет номер  70
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Псевдонимы

Представим, что нам нужно создать копию списка для дальнейшей его обработки. Если мы поступим как с неизменяемыми объектами (числами, строками и т.д.), то обнаружим неприятный эффект – изменения одного списка отражаются на содержимом другого списка:

```
>>> h = ['bonjour', 7, 'hola', -1.0, 'привет']
>>> p = h
>>> p
['bonjour', 7, 'hola', -1.0, 'привет']
>>> p[0] = 1
>>> p
[1, 7, 'hola', -1.0, 'привет']
>>> h
[1, 7, 'hola', -1.0, 'привет']
```



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Поверхностное копирование (1)

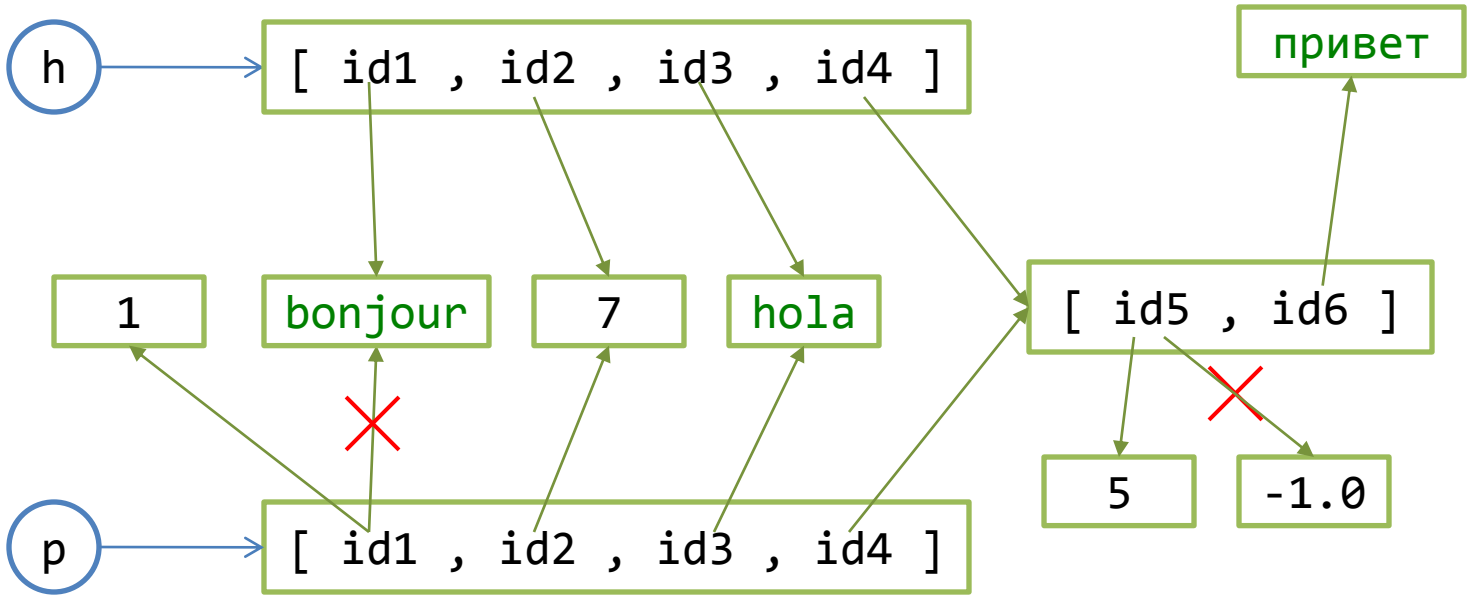
Для создания копии списка есть два способа. Первый называется поверхностным копированием. Он создаёт новый объект-список, который хранит в себе ссылки на объекты-элементы первого.

```
>>> h = ['bonjour', 7, 'hola', [-1.0, 'привет']]
>>> p = h[:] # создаём поверхностную копию
>>> p
['bonjour', 7, 'hola', [-1.0, 'привет']]
>>> p[0] = 1 # замена обычного объекта проходит хорошо
>>> p
[1, 7, 'hola', [-1.0, 'привет']]
>>> h
['bonjour', 7, 'hola', [-1.0, 'привет']]
>>>
>>> p[3][0] = 5 # элемента вложенного списка - уже нет
>>> p
[1, 7, 'hola', [5, 'привет']]
>>> h
['bonjour', 7, 'hola', [5, 'привет']]
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Поверхностное копирование (2)

Наглядно продемонстрировать проблему может следующая схема:



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Глубокое копирование (1)

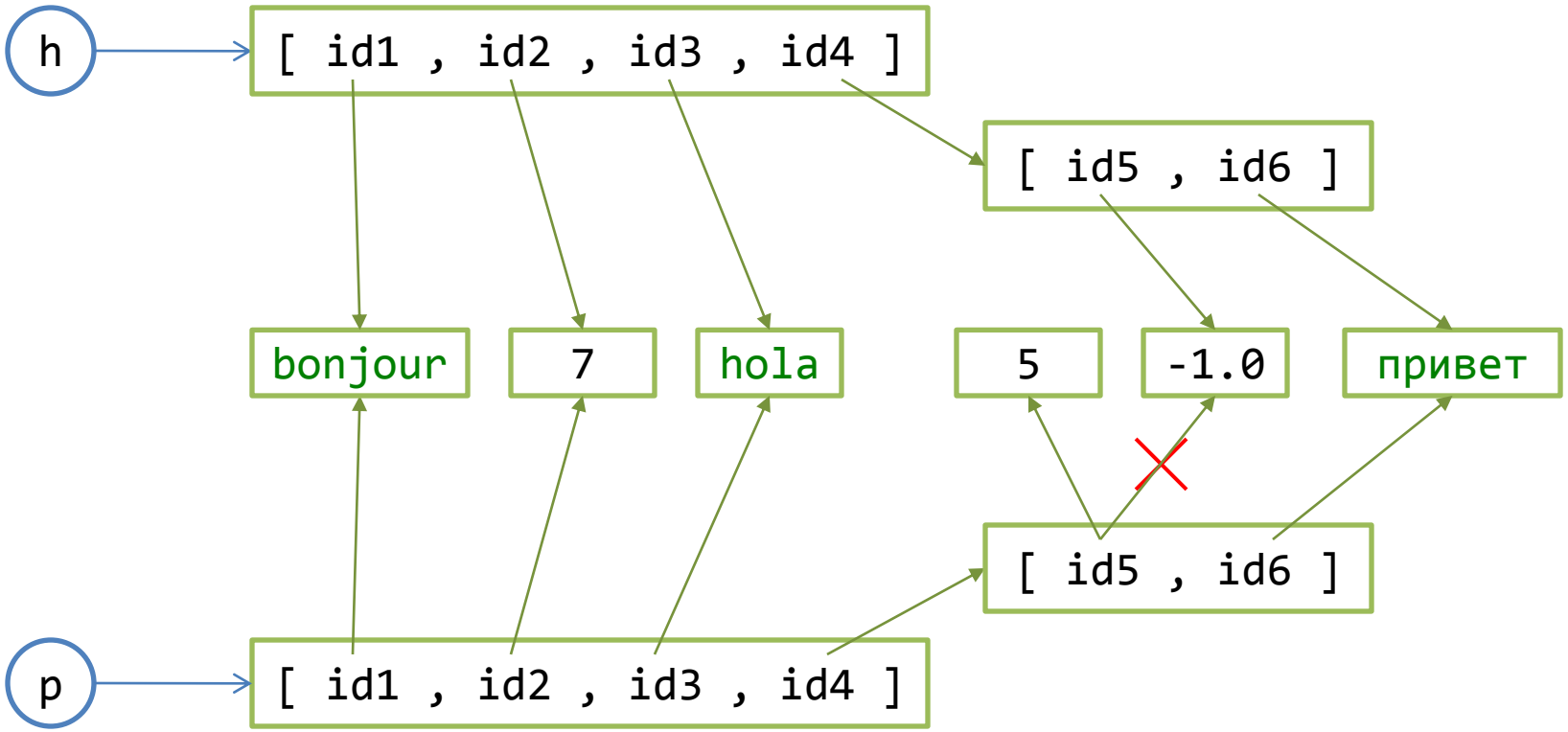
Для создания полной копии списка необходимо использовать глубокое копирование. В этом случае происходит рекурсивное создание всех объектов, содержащихся в исходном списке.

```
>>> from copy import deepcopy
>>>
>>> h = ['bonjour', 7, 'hola', [-1.0, 'привет']]
>>>
>>> p = deepcopy(h) # создаём полную копию
>>> p
['bonjour', 7, 'hola', [-1.0, 'привет']]
>>>
>>> p[3][0] = 5 # теперь всё работает нормально
>>> p
['bonjour', 7, 'hola', [5, 'привет']]
>>> h
['bonjour', 7, 'hola', [-1.0, 'привет']]
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Глубокое копирование (2)

Наглядно продемонстрировать глубокое копирование может следующая схема:



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева